🛠️ Ejercicios

Tabla de contenidos

1 Problemas con el descuento 1
2 Reportes 2
3 Notificaciones 2
4 Coordenadas 3
5 Terrenos 3
6 Compras 3
7 Tiempo 4
8 Contador 4
9 Magia para programadores 4
10 Mensaje secreto 5
10.1 Punto extra 6
11 Real envido 6
12 La muestra infinita 6
12.1 Punto extra 7
13 ¡Orden en el laboratorio! 7
13.1 Puntos extra 7
14 Sensores descalibrados 8
14.1 Jerarquía de clases 8
14.2 Función auxiliar 8
15 Python para matemáticos 9
15.1 Punto extra 9
16 Tiempo al tiempo ⏳ 9
16.1 Primera implementación 9
16.2 A prueba de balas 10
17 Buscando el error 12
18 Cada problema con su nombre 13
19 Un cajero insistente 13

1 Problemas con el descuento

El siguiente código calcula mal el precio final de un artículo con descuento. Localice el problema sin duplicar en la subclase la lógica de aplicar impuestos. Corríjalo usando `super()`.

```
class Articulo:
    def __init__(self, precio):
        self.precio = precio

    def precio_final(self):
```

```
        return self.precio * 1.21

class ArticuloEnOferta(Articulo):
    def __init__(self, precio, descuento):
        self.descuento = descuento

    def precio_final(self):
        return self.precio * (1 - self.descuento)
```

Para `ArticuloEnOferta(100, 0.20)`, ¿cuál debería ser el resultado si el descuento se aplica antes del impuesto? Escriba una prueba pequeña que lo compruebe.

2 Reportes

Considere una aplicación que exporta reportes:

```
class Reporte:
    def exportar(self):
        raise NotImplementedError

class ReporteTexto(Reporte):
    def exportar(self):
        return "texto plano"

class ReporteCSV(Reporte):
    def exportar(self):
        return "a,b,c"

def guardar_todos(reportes):
    return [reporte.exportar() for reporte in reportes]
```

Agregue una clase `ReporteJSON` sin modificar `guardar_todos`. Luego agregue por accidente un `str` a la lista de reportes: ¿qué error aparece y en qué lugar? Proponga una estrategia para detectar ese problema antes, ya sea con una clase base abstracta, anotaciones de tipo o una validación explícita. ¿Qué garantiza realmente cada alternativa durante la ejecución?

3 Notificaciones

Diseñe una jerarquía para un sistema de notificaciones que pueda enviar por correo, por consola y por un canal que todavía no existe. Defina el contrato mínimo de la clase base y una función `notificar_a_todos(notificadores, mensaje)` que aproveche polimorfismo. ¿Qué parte del diseño cambiaría si algunos notificadores pudieran fallar y otros no?

4 Coordenadas

Ejecute el siguiente código y explique cada resultado. Después implemente `__eq__` para que dos coordenadas con los mismos valores se consideren iguales, sin que una coordenada pueda ser igual a un objeto de una clase no relacionada.

```
class Coordenada:
    def __init__(self, x, y):
        self.x = x
        self.y = y

origen_a = Coordenada(0, 0)
origen_b = Coordenada(0, 0)
print(origen_a == origen_b)
print(origen_a is origen_b)
print(origen_a.__dict__ == origen_b.__dict__)
```

¿Qué debería devolver `Coordenada(0, 0).__eq__((0, 0))`? Pruébelo y justifique la decisión.

5 Terrenos

Una clase usa el área para ordenar terrenos. Complete `__lt__`, pero piense primero qué ocurriría con terrenos de igual área y distinta forma.

```
class Terreno:
    def __init__(self, frente, fondo):
        self.frente = frente
        self.fondo = fondo

    def area(self):
        return self.frente * self.fondo
```

Cree tres objetos, ordénelos con `sorted` y compare el criterio de orden con el de igualdad. ¿Es necesario que dos objetos con la misma área sean iguales? ¿Por qué una respuesta distinta a esta pregunta puede ser válida en otro dominio?

6 Compras

Diseñe una clase `Carrito` que haga que las siguientes expresiones sean coherentes:

```
len(carrito)
"yerba" in carrito
print(carrito)
```

Indique qué métodos mágicos implementaría y qué debería devolver cada uno. Decida también si `"yerba" in carrito` busca por nombre, por un objeto `Producto` completo o por ambas cosas; explique cómo esa decisión afecta el contrato de la clase.

7 Tiempo

Se quiere representar una duración en minutos mediante una clase `Duracion`. Proponga las implementaciones de `__add__`, `__repr__` y `__str__` para que se cumpla lo siguiente:

```
pausa = Duracion(15)
reunion = Duracion(45)
total = pausa + reunion
print(total)       # 1 h 0 min
repr(total)        # Duracion(60)
```

¿Debería permitirse `pausa + 10`? Elija una respuesta, impleméntela y describa qué error o comportamiento espera quien use la clase.

8 Contador

Defina una clase `Contador` que represente un contador numérico. Por defecto, las instancias comienzan con el valor `0`, aunque debe permitirse inicializarlas con un valor distinto.

Implemente los siguientes métodos:

- `incrementar`: aumenta el valor del contador en una cantidad arbitraria (por defecto, 1).
- `decrementar`: disminuye el valor del contador en una cantidad arbitraria (por defecto, 1).
- `reiniciar`: restablece el contador a su valor inicial (¡que puede ser distinto de 0!).
- `valor`: devuelve el valor actual del contador.

Ejemplo de uso

```
contador = Contador()
contador.incrementar(5)    # El valor interno es 5
contador.decrementar(2)    # El valor interno es 3
print(contador.valor())    # Imprime 3
contador.reiniciar()
print(contador.valor())    # Imprime 0
```

9 Magia para programadores

Esta es tu primera clase de Pociones en Hogwarts y el profesor te dio como tarea descubrir de qué color se volverá una poción si se mezcla con otra. Todas las pociones tienen un color definido en formato RGB, desde `[0, 0, 0]` hasta `[255, 255, 255]`.

Para complicar un poco más la tarea, el profesor realizará varias mezclas seguidas y luego te preguntará por el color final. Además del color, también deberás calcular qué volumen tendrá la poción después de la mezcla final.

Gracias a tu experiencia en programación descubriste que al mezclar dos pociones, los colores se combinan como si se mezclaran dos colores en formato RGB. Por ejemplo, si mezclas una poción con color `[255, 255, 0]` y volumen `10` con otra de color `[0, 254, 0]` y volumen `5`, obtendrás una nueva poción con:

- color `[170, 255, 0]`
- volumen `15`

Por lo tanto, decidís crear una clase `Pocion` que tenga:

- dos propiedades:
  - `color` (una lista o tupla con 3 enteros)
  - `volumen` (un número)
- un método `mezclar` que acepte otra `Pocion` y devuelva una nueva `Pocion` ya mezclada.

Ejemplo:

```
felix_felicis = Pocion([255, 255, 255],  7)
pocion_multijugos = Pocion([51, 102, 51], 12)
nueva_pocion = felix_felicis.mezclar(pocion_multijugos)

nueva_pocion.color # Devuelve [127, 159, 127]
nueva_pocion.volumen # Devuelve 19
```

> Ayuda
>
> Los colores de las pociones deben representarse como tríos de números enteros en formato RGB. Al realizar una mezcla de colores, se debe redondear hacia arriba utilizando `math.ceil`.

10 Mensaje secreto

Un **cifrado por sustitución simple** reemplaza cada carácter de un alfabeto con un carácter de un alfabeto alternativo. Cada posición en el alfabeto original se mapea a la posición correspondiente en el alfabeto alternativo, y esto sirve tanto para codificar como para decodificar.

El objetivo es crear una clase que, al inicializarse, reciba dos alfabetos (original y alternativo). La clase debe tener un método para encriptar mensajes y otro para revertir la encriptación.

Ejemplo:

```
alfabeto = "abcdefghijklmnopqrstuvwxyz"
alfabeto_mezclado = "etaoinshrdlucmfwypvbgkjqxz"

mi_cifrado = Cifrado(alfabeto, alfabeto_mezclado)

mi_cifrado.codificar("abc")    # => "eta"
mi_cifrado.codificar("xyz")    # => "qxz"
mi_cifrado.codificar("aeiou")  # => "eirfg"

mi_cifrado.decodificar("eta")    # => "abc"
mi_cifrado.decodificar("qxz")    # => "xyz"
mi_cifrado.decodificar("eirfg")  # => "aeiou"
```

10.1 Punto extra

Verifique en el método `__init__` que la longitud de los alfabetos sea la misma. Caso contrario, levante una excepción `ValueError` indicando cuál es el problema.

11 Real envido

Construir una clase `ManoDeTruco` que, al inicializarse, reciba una lista o tupla de hasta tres números enteros, correspondientes a los valores de las cartas en una mano de truco.

La clase debe incluir un método llamado `comparar_con` que recibe otra mano de truco y determine cuál de las dos suma más puntos para el envido. En caso de empate, se considera ganadora la mano que invocó el método.

Ejemplo de uso

```
mano1 = ManoDeTruco([7, 5, 6])
mano2 = ManoDeTruco([4, 11, 2])

ganadora = mano1.comparar_con(mano2)
```

Consideraciones

- Asuma que todas las cartas cargadas en la mano son del mismo palo.
- Para calcular los puntos del envido, solo se consideran dos de las tres cartas, no la suma de las tres.

12 La muestra infinita

Defina una clase `Muestra` que represente un conjunto de datos numéricos. La clase debe inicializarse a partir de un iterable de números e implementar los siguientes métodos:

- `agregar(x)`: agrega un número a la muestra.
- `n()`: devuelve la cantidad de elementos.
- `suma()`: devuelve la suma de los valores.
- `media()`: devuelve el promedio de los valores.
- `varianza(muestral=False)`: calcula la varianza.
  - Si `muestral=False`, se usa el denominador `n` (varianza poblacional).
  - Si `muestral=True`, se usa el denominador `n-1` (varianza muestral).

Ejemplo de uso

```
muestra = Muestra([10, 12, 13, 15])
muestra.agregar(20)
muestra.n()                    # 5
muestra.suma()                 # 70
muestra.media()                # 14.0
```

```
muestra.varianza()              # varianza poblacional
muestra.varianza(muestral=True) # varianza muestral
```

12.1 Punto extra

Modifique la clase para que:

- Guarde los datos en un atributo "privado" llamado `_datos`.
- Provea una propiedad de solo lectura `valores`, que devuelva una copia inmutable de los datos (por ejemplo, una tupla).

13 ¡Orden en el laboratorio!

En un laboratorio se necesita llevar un registro ordenado de los experimentos realizados. Cada experimento debe contar con un número identificador único, un nombre y, de manera opcional, el nombre de la persona responsable.

El objetivo de este ejercicio es construir una clase que facilite dicha organización.

Para ello, implemente una clase llamada `Experimento` que se inicializa con el nombre del experimento y, opcionalmente, con el nombre del responsable. La clase debe asignar automáticamente un número identificador único a cada instancia. Para lograrlo, utilice un **atributo de clase** llamado `total_creados`, que comience en 0.

Cada instancia debe contar con:

- Un identificador numérico único (asignado automáticamente de forma incremental por la clase).
- Un nombre.
- Un responsable, si se proporciona.

Ejemplo de uso

```
e1 = Experimento("Piloto A", responsable="Dolores")
e2 = Experimento("Piloto Z")
Experimento.total_creados  # Devuelve 2
```

13.1 Puntos extra

Modifique la clase para que también:

- Se puedan crear objetos utilizando un **método de clase** llamado `desde_dict` que reciba un diccionario de la forma `{"nombre": ..., "responsable": ...}` y devuelva una instancia de `Experimento`.
- Implemente el método mágico `__repr__` que devuelva una cadena de texto con el siguiente formato: `python Experimento(id=1, nombre="A/B", responsable="Sosa")`

Ejemplo de uso

```python
e = Experimento.desde_dict({"nombre": "Piloto B", "responsable": "Ana"})
repr(e)
# Experimento(id=3, nombre="Piloto B", responsable="Ana")
```

14 Sensores descalibrados

Este ejercicio requiere diseñar una pequeña jerarquía de clases para simular sensores utilizados en un laboratorio.

En la práctica, los sensores no siempre son completamente precisos: pueden registrar valores ligeramente superiores o inferiores al valor real. Para corregir ese desvío, se aplica una **calibración**, que consiste en ajustar las lecturas mediante un valor adicional o corrector llamado *offset*.

14.1 Jerarquía de clases

Clase base: `Sensor`

La clase `Sensor` debe incluir:

- Un atributo `nombre` para identificar al sensor.
- Un método `leer` que levante una excepción `NotImplementedError`, indicando que debe ser implementado por las subclases.
- Un método `calibrar(offset)` que permita almacenar un valor de ajuste (`offset`) que se aplicará a las lecturas.

Subclases

Se deben definir dos subclases: `SensorTemperatura` y `SensorHumedad`, ambas con su propia implementación del método `leer`:

- `SensorTemperatura`: simula una medición utilizando `random.uniform(18, 28)`.
- `SensorHumedad`: simula una medición utilizando `random.uniform(30, 70)`.

En ambos casos, la medición debe ser ajustada por el *offset* correspondiente (si fue calibrado).

14.2 Función auxiliar

Además, implemente una función llamada `promedio_lecturas` que reciba dos argumentos: una secuencia de sensores y un número entero `n`, que indica cuántas lecturas realizar con cada sensor.

La función debe realizar `n` lecturas para cada sensor utilizando su método `leer`, calcular el promedio de esas lecturas y devolver un diccionario que asocie el nombre de cada sensor con su promedio correspondiente.

Ejemplo de uso

```python
st = SensorTemperatura("T1")
sh = SensorHumedad("H1")
st.calibrar(0.5)
```

```
promedios = promedio_lecturas([st, sh], n=3)
# {'T1': ..., 'H1': ...}
```

15 Python para matemáticos

Construya una clase `Fraccion` que acepte dos argumentos: numerador y denominador. Se desea que esta clase:

a. Sea representable como cadena de texto.
b. Implemente la suma entre fracciones.
c. Devuelva siempre el resultado en la **mínima representación posible** (fracción irreducible).

Ejemplo:

```
fraccion1 = Fraccion(4, 5)
print(fraccion1 + Fraccion(1, 8))
#> "37/40"
```

15.1 Punto extra

Extender la funcionalidad de la clase incluyendo las operaciones de resta, multiplicación y división.

16 Tiempo al tiempo ⏳

El módulo estándar `datetime` de Python incluye, entre otras cosas, la clase `date` para trabajar con fechas. El objetivo de este ejercicio es implementar, desde cero, una nueva clase `Date` que permita representar fechas y operar con ellas.

16.1 Primera implementación

1. Construya la clase `Date`. El método `__init__` debe tomar 3 parámetros: `year`, `month` y `day`, y asignarlos como atributos de la instancia. Cree un objeto que represente el 6 de noviembre de 1995 y verifique que los atributos contengan los valores esperados.
2. Implemente el método `__str__`, que debe devolver una cadena con el formato `YYYY-MM-DD`, donde `YYYY` representa el año, `MM` el mes y `DD` el día. **Ayuda**: Puede utilizar el método `.rjust` de las cadenas de texto para agregar ceros a la izquierda cuando sea necesario. Por ejemplo, para el 6 de noviembre de 1995, `str(fecha)` debe devolver `"1995-11-06"`.
3. Implemente el método `__repr__`, que debe devolver una representación similar a la utilizada para crear la instancia. Para la fecha 6 de noviembre de 1995, debe devolver la cadena `"Date(year=1995, month=11, day=6)"`.
4. Implemente un método de clase que permita crear una `Date` a partir de una cadena en formato `YYYY-MM-DD`. Por ejemplo, `Date.from_str("2025-03-08")` debe devolver un objeto `Date` que representa al 8 de marzo de 2025. **Ayuda**: Puede utilizar el método `.lstrip("0")` para eliminar ceros a la izquierda en una cadena.
5. Implemente los métodos de comparación entre objetos `Date`.

i. `__eq__`: igualdad (==)
ii. `__ne__`: desigualdad (!=)
iii. `__lt__`: menor que (<)
iv. `__gt__`: mayor que (>)
v. `__le__`: menor o igual que (<=)
vi. `__ge__`: mayor o igual que (>=)

Ayuda: Dos fechas se consideran iguales si sus atributos `year`, `month`, y `day` coinciden. Para las comparaciones, considere primero el año, luego el mes y finalmente el día.

16.2 A prueba de balas

A esta altura, se cuenta con una implementación razonablemente completa y funcional para representar objetos de tipo fecha. Sin embargo, la clase `Date` no garantiza la validez ni la robustez de las instancias que se crean. Es posible construir objetos `Date` que representen fechas inválidas, como el 31 de abril o el 55.8 del mes 25.3, y operar con ellos sin que el programa emita ningún tipo de advertencia.

6. Modifique la implementación de los atributos `year`, `month` y `day` de modo que sean privados y que su asignación incluya una verificación de validez. Para ello, se propone lo siguiente:
   i. Utilice atributos privados llamados `_year`, `_month` y `_day`.
   ii. Defina métodos `year`, `month` y `day`, decorados con `@property`, que expongan dichos atributos para su lectura. Estos métodos deben devolver el valor del atributo correspondiente.
   iii. Para permitir la asignación controlada, implemente un *setter* para cada atributo Para ello, decore el mismo método con `@<nombre>.setter` (por ejemplo, `@year.setter`) y defina allí la lógica de verificación. Por el momento, verifique únicamente que el valor asignado sea un número entero y positivo. Eleve un `ValueError` en caso de que el valor por asignar no pase la verificación.

Utilice los siguientes ejemplos de verificación

```
d = Date(2022, 7, 1)
repr(d) # Date(year=2022, month=7, day=1)
str(d)  # "2022-07-01"
(d.year, d.month, d.day) # (2022, 7, 1)
```

```
Date(2022, 0, 1)       # Falla: el mes no es positivo
Date(2022, 3, 5.5)     # Falla: el día no es un entero
Date("11", 3, 1)       # Falla: el año es una cadena de texto
```

```
d = Date(2022, 7, 1)
d.day = 11             # Funciona: pasa la verificación
str(d)                 # "2022-07-11"
```

```
d.month = -1            # Falla
d.year = -1             # Falla
```

7. Mejore ahora las verificaciones de los valores asignados para que sean más robustas:
   - El año debe ser entero y mayor o igual a 0.
   - El mes debe ser un número entero entre 1 y 12 inclusive.
   - El día debe ser un número entero mayor o igual a 1 y menor o igual a la cantidad de días del mes correspondiente.

La cantidad de días depende del mes. Para todos los meses excepto febrero, utilice el diccionario provisto en la ayuda debajo. Para el caso de febrero, implemente un método estático que reciba un año como argumento y devuelva un booleano indicando si ese año es bisiesto.

Ayuda

Diccionario de días por mes:

```
DOM = {
    1: 31,  # Enero
    2: 28,  # Febrero (29 en año bisiesto)
    3: 31,  # Marzo
    4: 30,  # Abril
    5: 31,  # Mayo
    6: 30,  # Junio
    7: 31,  # Julio
    8: 31,  # Agosto
    9: 30,  # Septiembre
    10: 31, # Octubre
    11: 30, # Noviembre
    12: 31, # Diciembre
}

DOM[5] # Devuelve 31, porque es mayo
```

Por otro lado, un año es bisiesto si:

- Es divisible por 4,
- excepto que sea divisible por 100,
- salvo que tambien sea divisibles por 400

En Python

```
(year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

Finalmente, verifique que su clase `Date` funciona correctamente con los siguientes ejemplos:

```
# Definición de fechas
d1 = Date(1990, 5, 20)
d2 = Date(1998, 9, 21)
d3 = Date(1998, 9, 20)
d4 = Date(2022, 12, 18)

d1 < d2
d1 > d2
d1 != d3
d4 >= d3

Date(2000, 2, 29) # Funciona, es bisiesto
Date(2001, 2, 29) # Falla, no es bisiesto


l = [d1, d2, d3, d4]
sorted(l) # Funciona, ¿por qué?
```

17 Buscando el error

Cada uno de los siguientes programas contiene un error diferente. Sin ejecutarlos, indique qué tipo de error espera encontrar, en qué línea se origina y cuál es su causa. Luego ejecute cada fragmento por separado, lea el mensaje completo y compruebe su respuesta.

Finalmente, corrija cada programa mediante el cambio más pequeño posible. No utilice `try` y `except`: en estos casos corresponde solucionar el error que existe en el código.

```
nombre = "Ada"
print(nomrbe)
```

```
edades = [19, 21, 23]
ultima_edad = edades[3]
print(ultima_edad)
```

```
total = 1500
print("Total: $" + total)
```

```
temperaturas = (18.5, 19.2, 20.0)
temperaturas.append(20.4)
```

```
def saludar(nombre)
    return f"¡Hola, {nombre}!"
```

Para cada caso, explique qué información del mensaje de error fue útil para encontrar el problema.

18 Cada problema con su nombre

Implemente una clase `Inventario` que represente las existencias de productos de un comercio. Su constructor debe recibir un diccionario cuyas claves sean nombres de productos y cuyos valores sean cantidades disponibles. Guarde una copia de ese diccionario en un atributo llamado `stock`.

El constructor debe verificar los datos recibidos y lanzar una excepción apropiada cuando:

- el argumento no sea un diccionario;
- el nombre de algún producto no sea una cadena de texto no vacía;
- alguna cantidad no sea un número entero mayor o igual que cero.

La clase debe implementar además un método `hay_stock(producto, cantidad)`. Este método debe verificar que `producto` sea una cadena no vacía, que `cantidad` sea un número entero mayor que cero y que el producto exista en el inventario. Si todos los datos son válidos, debe devolver `True` cuando exista suficiente stock y `False` en caso contrario.

Distinga cuidadosamente los casos que corresponden a `TypeError`, `ValueError` y `KeyError`, e incluya en cada excepción un mensaje que describa el problema.

Utilice estas llamadas para probar la clase:

```
inventario = Inventario({"cuadernos": 12, "lápices": 30, "carpetas": 5})

inventario.hay_stock("cuadernos", 8)   # True
inventario.hay_stock("carpetas", 9)    # False
inventario.hay_stock("", 2)            # Falla: producto vacío
inventario.hay_stock("reglas", 1)      # Falla: producto inexistente
inventario.hay_stock("lápices", 2.5)   # Falla: cantidad de tipo incorrecto
inventario.hay_stock("lápices", -2)    # Falla: cantidad inválida

Inventario(["cuadernos", "lápices"])  # Falla: no es un diccionario
Inventario({"cuadernos": -3})         # Falla: cantidad inválida
```

19 Un cajero insistente

Este ejercicio debe resolverse como un programa interactivo completo. Escriba todo el código en un archivo llamado `cajero.py` y ejecútelo desde una terminal.

Defina dos excepciones personalizadas llamadas `MontoInvalidoError` y `SaldoInsuficienteError`. Ambas deben heredar de una excepción común llamada `CuentaError`.

Luego implemente una clase `CuentaBancaria` con un atributo `saldo` y un método `extraer(monto)`. El método debe:

- lanzar `MontoInvalidoError` si el monto no es mayor que cero;
- lanzar `SaldoInsuficienteError` si el monto supera el saldo disponible;
- descontar el monto y devolverlo cuando la extracción sea válida.

Finalmente, escriba una función `solicitar_extraccion(cuenta)` que solicite un monto mediante `input`. Utilice un bucle `while True` para repetir la solicitud cuando el texto ingresado no pueda convertirse a número o cuando la cuenta rechace la operación. El bucle debe terminar solamente después de realizar una extracción válida.

Pruebe el programa ingresando, en este orden, un texto que no sea numérico, un monto negativo, un monto superior al saldo y un monto válido. El mensaje mostrado en cada intento debe permitir distinguir claramente qué problema ocurrió.

Al final de `cajero.py`, cree una cuenta con un saldo inicial, llame a `solicitar_extraccion` y muestre el saldo restante cuando la operación termine. Para ejecutar el programa, ubíquese desde la terminal en la carpeta donde guardó el archivo y escriba:

```
python cajero.py
```